

Project Interim Report
On
**Bank Account Management System
(Using Core Java & OOP Concepts)**

RU-100-01-00012
Object Oriented Programming with Java

SESSION: 2025-26



Submitted By
Aditi Sahu
ERP ID.: RU-25-10076

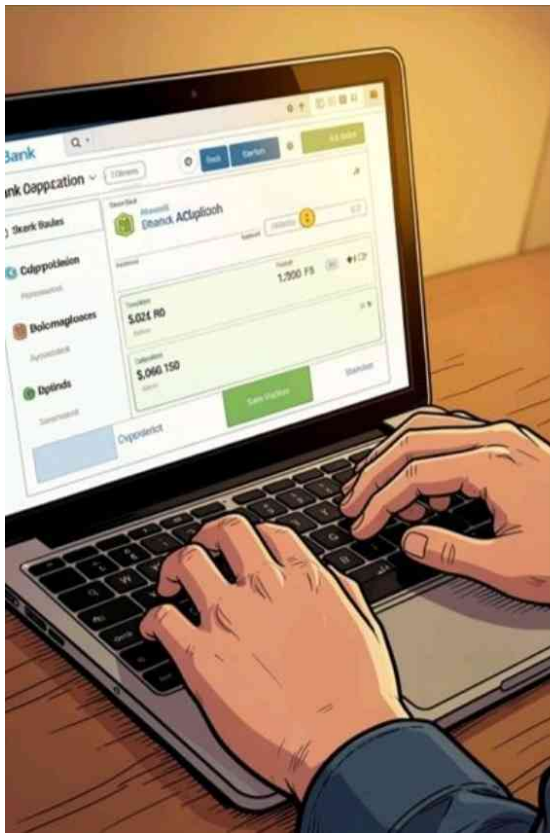
**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with IBM**

Under the Guidance of
Mr. Santanu Sasmal
IBM SME and EXPERT

**SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
RUNGTA INTERNATIONAL SKILLS UNIVERSITY**

BHILAI, CG

(April 2026)



Bank Account Management System — Project Overview

A compact, object-oriented Java project that models secure bank account operations. Students implement a `BankAccount` class to safely manage balances, perform deposits and withdrawals with validation, and simulate transactions — demonstrating encapsulation, defensive programming, and object interaction.

Why this project matters



Security-first thinking

Practices defensive checks to prevent invalid or malicious transactions.



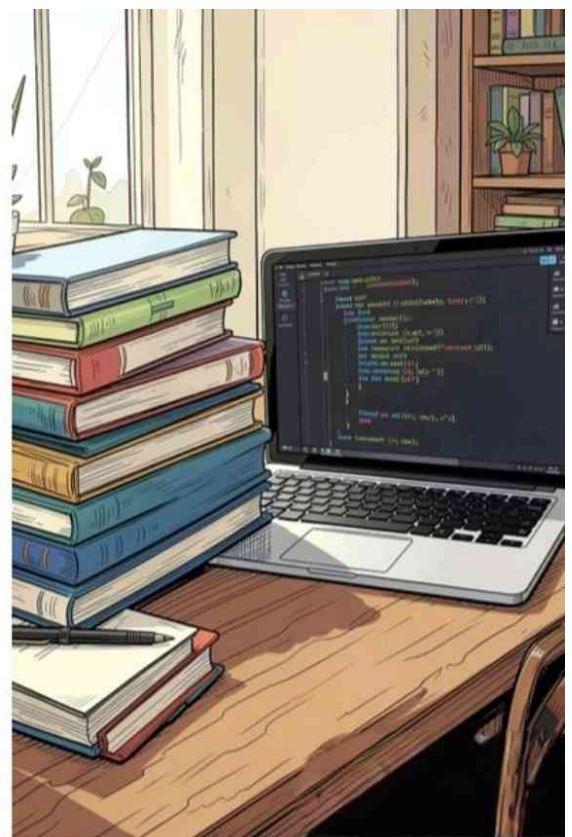
Core OOP skills

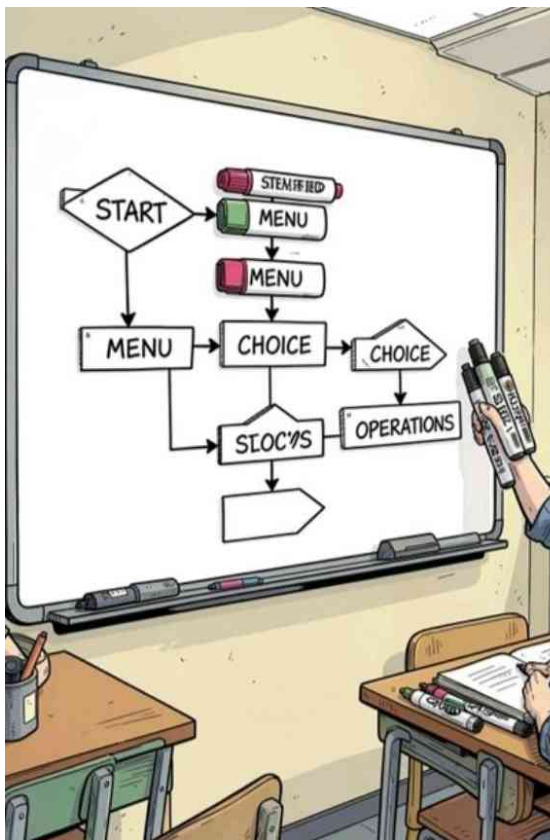
Applies encapsulation, single-responsibility methods, and clear object interfaces.



Extendable design

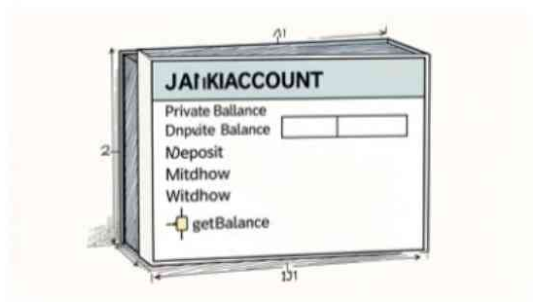
Clean class boundaries allow easy extension to databases, GUIs, or multi-user systems.





Project Objectives

- Securely manage account balance with private state and controlled access.
- Implement deposit() and withdraw() with robust validation and clear return/error messages.
- Prevent invalid transactions (negative amounts, overdrafts).
- Demonstrate encapsulation and object interaction in Java.



Class Diagram — BankAccount

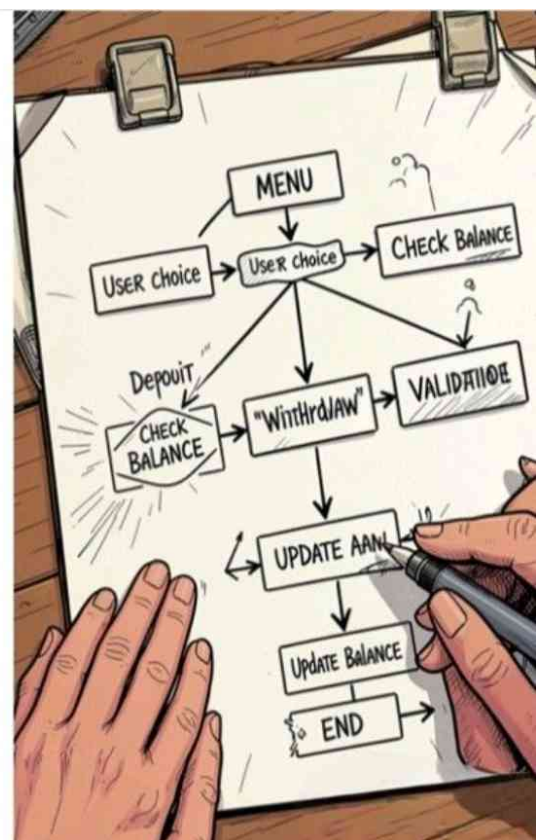
BankAccount

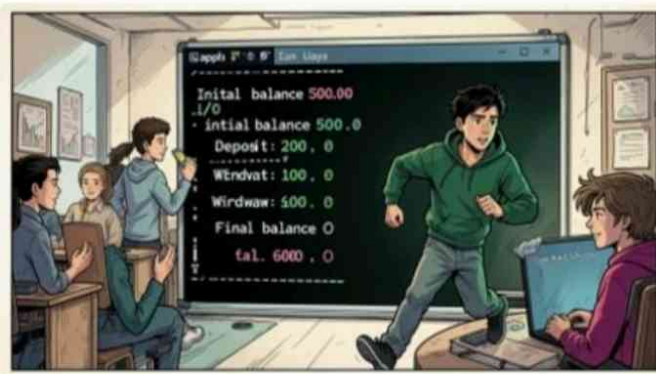
- Variables: **private double balance**
- Methods:
 - **deposit(double amount)** — validate amount > 0, add to balance
 - **withdraw(double amount)** — validate amount > 0 and amount ≤ balance
 - **getBalance()** — returns current balance (read-only)

System Flow — Step by Step



The interactive loop presents a menu, executes the requested operation with validation, updates state only on success, and displays clear success/error messages.





Sample I/O Scenarios

Valid flow:

- Initial Balance: 500.00 → deposit(200.00) → withdraw(100.00) → Final Balance: 600.00

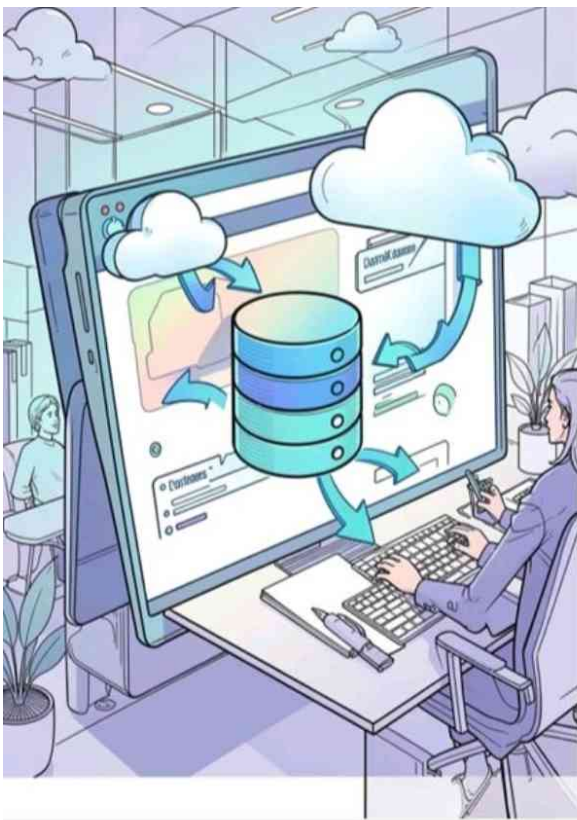
Invalid flow:

- withdraw(700.00) → "Withdrawal failed: Insufficient balance"
- deposit(-50.00) → "Deposit failed: Amount must be positive"


```
class BankAccount {  
    private double balance;  
  
    BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    double getBalance() {  
        return balance;  
    }  
  
    void deposit(double amount) {  
        if (amount > 0) balance +=  
amount;  
        else System.out.println("Deposit  
failed");  
    }  
  
    void withdraw(double amount) {  
        if (amount > 0 && amount <=  
balance) balance -= amount;  
        else  
System.out.println("Withdrawal failed");  
    }  
}
```

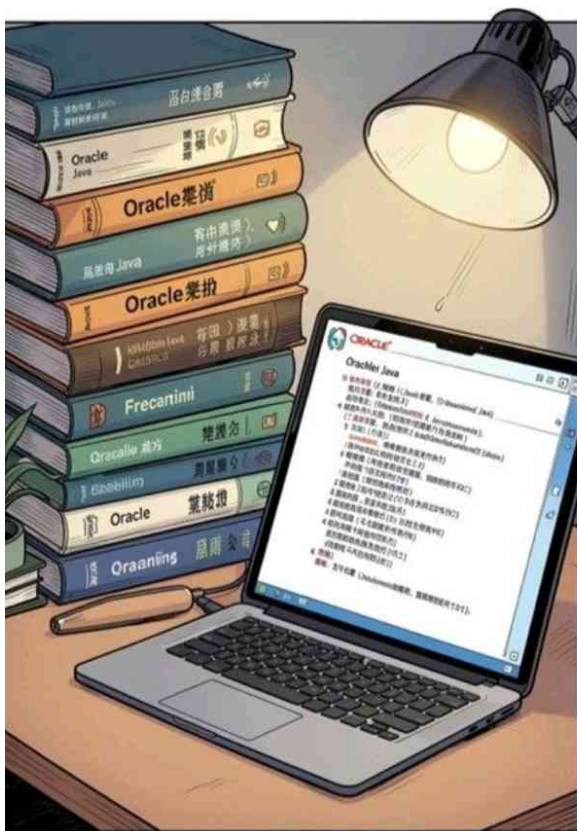


```
public class Main {  
    public static void main(String[]  
args) {  
        BankAccount acc = new  
BankAccount(500);  
        acc.deposit(200);  
        acc.withdraw(100);  
        System.out.println("Final  
Balance: " + acc.getBalance());  
    }  
}
```



Future Enhancements

- Database integration (JDBC/ORM) for persistent accounts and transaction logs.
- Multi-user support with authentication and session management.
- Thread-safe operations and concurrency control for simultaneous transactions.
- GUI (JavaFX/Swing) or web front-end for improved usability and demos.
- Audit trail, logging, and unit tests for reliability and grading criteria.



Conclusion & References

This project gives students a compact, practical exercise in OOP: designing a secure `BankAccount` class, enforcing validation, and simulating transactions. It reinforces encapsulation, method design, and system thinking — and provides a solid base for more advanced features.



Primary Reference

H. Schildt — *Java: The Complete Reference*



Online

Oracle Java Documentation — API references and best practices